

 Optimizers

বিস্তারিত বাংলা ML নোট | Intuition · Math · Code · Diagrams

 তৈরির তারিখ: ২০২৬-০৪-০৪ | সম্পূর্ণ বাংলায় লেখা ML Notes **Optimizers (অপটিমাইজার) — সম্পূর্ণ বাংলা নোট****সাবজেক্ট:** Machine Learning & Deep Learning**লেভেল:** Intermediate**পূর্বশর্ত:** Gradient Descent, Backpropagation, Neural Networks**তারিখ:** ২০২৬-০৪-০৯**১. 🎯 সহজ ভাষায় পরিচিতি (Intuition)****🌐 বাস্তব জীবনের উদাহরণ: পাহাড় থেকে নামা**

কল্পনা করো তুমি একটি পাহাড়ের চূড়ায় আছো এবং চোখে পড়ি বাঁধা। তোমার লক্ষ্য হলো সবচেয়ে নিচু জায়গায় (উপত্যকায়) পৌঁছানো।

এখন তুমি কীভাবে নামবে?

- **পা দিয়ে মাটি পরীক্ষা করো** — কোন দিকে ঢাল বেশি আছে?
- **সেই দিকে পা বাড়ানো** — ঢালের দিকে একটু একটু এগিয়ে যাও
- **বারবার পরীক্ষা করো** — প্রতিটি পদক্ষেপের পর আবার ঢাল দেখো

Machine Learning-এ **Loss Function** হলো সেই পাহাড়, **Weights** হলো তোমার অবস্থান, আর **Optimizer** হলো সেই বুদ্ধিমান পদক্ষেপের কৌশল যেটা ঠিক করে তুমি কতটুকু, কোন দিকে পা বাড়াবে।

🔥 আরেকটি উদাহরণ: রান্নায় লবণ ঠিক করা

ধরো তুমি প্রথমবার রান্না করছো। খাবারে লবণ ঠিকমতো দিতে হবে:

পরিস্থিতি	Optimizer-এর সমতুল্য
প্রতিবার অনেক বেশি লবণ দাও → ঝাল হয়ে যাচ্ছে	Learning Rate বড়
প্রতিবার একটু একটু করে দাও → সারাদিন লাগে	Learning Rate ছোট
আগের অভিজ্ঞতা মাথায় রেখে সঠিক পরিমাণ দাও	Adam Optimizer
প্রতিটি মশলার জন্য আলাদা নিয়ম → কিছু মশলা কম দরকার	AdaGrad Optimizer

? Optimizer কী সমস্যা সমাধান করে?

Neural Network ট্রেন করতে গেলে **Loss কমাতে** হয়। এই কাজ করে **Gradient Descent** — কিন্তু সাধারণ Gradient Descent অনেক সমস্যায় পড়ে:

- 1. **Local Minima** — পাহাড়ের একটা ছোট গর্তে আটকে যায়, আসল গভীরতম জায়গায় পৌঁছায় না
- 2. **Saddle Point** — সমতল জায়গায় আটকে যায় যেখানে Gradient ≈ 0
- 3. **Slow Convergence** — অনেক ধীরে ধীরে এগোয়
- 4. **Oscillation** — কান্ডিফ্রত জায়গার কাছে এসে ডানে-বামে দুলতে থাকে

Optimizers এই সব সমস্যা সমাধান করে বিভিন্ন কৌশলে!

২. 📖 বিস্তারিত ব্যাখ্যা (Deep Explanation)

🔑 মূল ধারণাগুলো (Core Concepts)

ক) Gradient (গ্র্যাডিয়েন্ট) কী?

Gradient হলো Loss Function-এর **ঢাল** (slope)। এটা বলে দেয় কোন ওজন (Weight) বাড়ালে Loss বাড়বে বা কমবে। আমরা সবসময় **Gradient-এর বিপরীত দিকে** যাই কারণ আমরা Loss কমাতে চাই।

Gradient $> 0 \rightarrow$ Loss বাড়ছে $\rightarrow$ Weight কমাও
 Gradient $< 0 \rightarrow$ Loss কমছে $\rightarrow$ Weight বাড়াও
 Gradient $= 0 \rightarrow$ Saddle/Minimum পয়েন্ট (সতর্ক থাকো!)

খ) Learning Rate (α) কী?

Learning Rate হলো প্রতিটি পদক্ষেপের **সাইজ** — কতটুকু পরিমাণে Weight আপডেট করব।

$\alpha = 0.01 \rightarrow$ ছোট পদক্ষেপ, ধীরে কিন্তু নিখুঁত
 $\alpha = 1.0 \rightarrow$ বড় পদক্ষেপ, দ্রুত কিন্তু লক্ষ্য ছাড়িয়ে যেতে পারে
 $\alpha = 0.001 \rightarrow$ অনেক ছোট, কখনো না পৌঁছানোর ভয়

গ) Batch কী?

ধরন	ব্যাখ্যা	সুবিধা	অসুবিধা
Full Batch GD	সব ডেটা একবারে ব্যবহার	Stable	অনেক মেমরি, ধীর
Mini-Batch GD	ছোট ব্যাচে ডেটা ভাগ (16, 32, 64)	Balance	কিছুটা Noisy
Stochastic GD	প্রতিটি sample আলাদাভাবে	দ্রুত	অনেক Noisy

🔧 প্রধান Optimizers বিস্তারিত

১। SGD (Stochastic Gradient Descent) — মূল ভিত্তি

সবচেয়ে সহজ Optimizer। প্রতিটি পদক্ষেপে Loss-এর Gradient হিসাব করে Weight আপডেট করে।

কাজের ধাপ:

1. একটি Mini-Batch নাও
2. সেই Batch দিয়ে Loss হিসাব করো
3. Gradient বের করো (Backpropagation)
4. Weight আপডেট করো: $W = W - \alpha \times \nabla L$
5. আবার ধাপ ১ থেকে

সমস্যা: Gradient zigzag করে, স্থির পথে এগোতে পারে না।

২। SGD with Momentum — গতিশক্তি যোগ করা

একটি বলকে পাহাড়ের নিচে গড়িয়ে দিলে সে আগের গতিশক্তি নিয়ে এগোয়। Momentum ঠিক তাই করে।

কাজের ধাপ:

1. আগের গতির (velocity) একটা অংশ রাখো
2. নতুন Gradient যোগ করো
3. সেই দিকে এগোও

সুবিধা: Oscillation কমে, Saddle Point পার হওয়া সহজ হয়।

৩। AdaGrad — প্রতিটি Parameter-এর জন্য আলাদা Learning Rate

ধরো একটি ছাত্র কিছু বিষয়ে (rare features) কম শিখেছে, কিছু বিষয়ে বেশি। AdaGrad বলে — যে বিষয়ে কম শিখেছে সেটা বেশি মনোযোগ দাও, যেটা অনেক দেখেছে সেটা কম চর্চা করলেও চলবে।

কাজের ধাপ:

1. প্রতিটি Parameter-এর Gradient-এর বর্গ জমা করতে থাকো
2. যে Parameter-এ বেশি Gradient হয়েছে, তার Learning Rate কমিয়ে দাও
3. কম Gradient → বড় Learning Rate (বেশি আপডেট)

সমস্যা: সময়ের সাথে সব Parameter-এর Learning Rate শূন্য হয়ে যায় → Training বন্ধ হয়ে যায়!

৪। RMSProp — AdaGrad-এর সমস্যা সমাধান

AdaGrad-এর মতোই কিন্তু পুরানো Gradient-গুলো ধীরে ধীরে ভুলে যায় (Exponential Moving Average)।

কাজের ধাপ:

1. Gradient² এর Moving Average রাখো (পুরনোটা ক্ষয় হয়)
2. সেই Moving Average দিয়ে Learning Rate ভাগ করো
3. Weight আপডেট করো

সুবিধা: Learning Rate কখনো পুরোপুরি শূন্য হয় না!

৫। Adam (Adaptive Moment Estimation) — সর্বোত্তম Optimizer

Adam হলো **Momentum + RMSProp** এর সমন্বয়। এটি দুটো জিনিস ট্র্যাক করে:

- **m (1st Moment):** Gradient-এর Exponential Moving Average (গতির দিক)
- **v (2nd Moment):** Gradient²-এর Exponential Moving Average (গতির শক্তি)

এবং **Bias Correction** ব্যবহার করে শুরুতে ভুল না হওয়ার জন্য।

কাজের ধাপ:

1. Gradient হিসাব করো: g_t
2. 1st Moment আপডেট: $m_t = \beta_1 \times m_{t-1} + (1-\beta_1) \times g_t$
3. 2nd Moment আপডেট: $v_t = \beta_2 \times v_{t-1} + (1-\beta_2) \times g_t^2$
4. Bias Correct করো: $\hat{m} = m_t / (1-\beta_1^t)$, $\hat{v} = v_t / (1-\beta_2^t)$
5. Weight আপডেট: $W = W - \alpha \times \hat{m} / (\sqrt{\hat{v}} + \epsilon)$

৬। AdamW — Modern Best Practice

Adam-এর একটি উন্নত সংস্করণ যেখানে **Weight Decay** সঠিকভাবে আলাদা করা হয়েছে। Transformer এবং আধুনিক LLM-গুলো এটি ব্যবহার করে।

৩. Math / Theory

SGD এর মূল সমীকরণ

$$W(t+1) = W(t) - \alpha \times \nabla L(W(t))$$

Symbol ব্যাখ্যা:

- $W(t)$ = t-তম iteration-এ Weight
- α = Learning Rate (সাধারণত 0.001 - 0.1)
- $\nabla L(W)$ = Loss-এর Gradient (ঢাল)

SGD with Momentum

$$v(t) = \beta \times v(t-1) + (1-\beta) \times \nabla L(W)$$

$$W(t+1) = W(t) - \alpha \times v(t)$$

- v = Velocity (গতিবেগ)
- β = Momentum coefficient (সাধারণত 0.9)

AdaGrad

$$G(t) = G(t-1) + [\nabla L(W)]^2 \quad \leftarrow \text{Gradient বর্গ জমা}$$

$$W(t+1) = W(t) - (\alpha / \sqrt{G(t) + \epsilon}) \times \nabla L(W)$$

- $G(t)$ = t পর্যন্ত সব Gradient-এর বর্গের যোগফল
- ϵ = Epsilon (ছোট সংখ্যা যাতে ভাগ শূন্য না হয়, সাধারণত $1e-8$)

RMSProp

$$v(t) = \beta \times v(t-1) + (1-\beta) \times [\nabla L(W)]^2 \quad \leftarrow \text{Moving Average}$$

$$W(t+1) = W(t) - (\alpha / \sqrt{v(t) + \epsilon}) \times \nabla L(W)$$

- β = Decay rate (সাধারণত 0.9)

Adam — সম্পূর্ণ সমীকরণ

$$g(t) = \nabla L(W(t)) \quad \leftarrow \text{Step 1: Gradient}$$

$$m(t) = \beta_1 \times m(t-1) + (1-\beta_1) \times g(t) \quad \leftarrow \text{Step 2: 1st Moment}$$

$$v(t) = \beta_2 \times v(t-1) + (1-\beta_2) \times g(t)^2 \quad \leftarrow \text{Step 3: 2nd Moment}$$

$$\hat{m}(t) = m(t) / (1 - \beta_1^t) \quad \leftarrow \text{Step 4: Bias Correction}$$

$$\hat{v}(t) = v(t) / (1 - \beta_2^t)$$

$$W(t+1) = W(t) - \alpha \times \hat{m}(t) / (\sqrt{\hat{v}(t) + \epsilon}) \quad \leftarrow \text{Step 5: Update}$$

Default Hyperparameters:

- $\alpha = 0.001$
- $\beta_1 = 0.9$ (1st Moment decay)
- $\beta_2 = 0.999$ (2nd Moment decay)
- $\epsilon = 1e-8$

Manual Calculation (ছোট উদাহরণ)

ধরো: $W = 2.0$, $\alpha = 0.1$, $g = 0.5$ (Gradient), $\beta_1 = 0.9$, $\beta_2 = 0.999$

SGD:

$$W_{\text{new}} = 2.0 - 0.1 \times 0.5 = 2.0 - 0.05 = 1.95$$

Adam ($t=1$, $m_0=0$, $v_0=0$):

$$m_1 = 0.9 \times 0 + 0.1 \times 0.5 = 0.05$$

$$v_1 = 0.999 \times 0 + 0.001 \times 0.25 = 0.00025$$

$$\hat{m}_1 = 0.05 / (1 - 0.9^1) = 0.05 / 0.1 = 0.5$$

$$\hat{v}_1 = 0.00025 / (1 - 0.999^1) = 0.00025 / 0.001 = 0.25$$

$$\begin{aligned} W_{\text{new}} &= 2.0 - 0.001 \times 0.5 / (\sqrt{0.25} + 1e-8) \\ &= 2.0 - 0.001 \times 0.5 / 0.5 \\ &= 2.0 - 0.001 = 1.999 \end{aligned}$$

পর্যবেক্ষণ: Adam অনেক ছোট পদক্ষেপ নেয় (বেশি নিরাপদ) কিন্তু সঠিক দিকে!

8. Code Example (Python)

```

# =====
# ML Optimizers – Complete Python Implementation
# =====

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.preprocessing import StandardScaler
import tensorflow as tf
from tensorflow import keras

# -----
# PART 1: NumPy দিয়ে Optimizer হাতে তৈরি করা
# -----

# সহজ Quadratic Function:  $f(x) = x^2 + 2x + 1$ 
# এর Minimum আছে  $x = -1$  তে

def loss_fn(x):
    """Loss Function:  $f(x) = x^2 + 2x + 1$ """
    return x**2 + 2*x + 1

def gradient(x):
    """Loss-এর Gradient:  $df/dx = 2x + 2$ """
    return 2*x + 2

# — 1. SGD -----
def sgd_optimizer(start_x, lr=0.1, epochs=50):
    """Stochastic Gradient Descent"""
    x = start_x                # শুরুর অবস্থান
    history = [x]              # পথ রেকর্ড করব

    for i in range(epochs):
        grad = gradient(x)     # Gradient বের করো
        x = x - lr * grad      # Weight আপডেট:  $W = W - \alpha \times \nabla L$ 
        history.append(x)

    return history

# — 2. SGD with Momentum -----

```



```

def momentum_optimizer(start_x, lr=0.1, beta=0.9, epochs=50):
    """SGD with Momentum"""
    x = start_x                # শুরু অবস্থান
    v = 0                      # শুরুতে Velocity শূন্য
    history = [x]

    for i in range(epochs):
        grad = gradient(x)     # Gradient বের করো
        v = beta * v + (1 - beta) * grad  # Velocity আপডেট
        x = x - lr * v         # Position আপডেট
        history.append(x)

    return history

# — 3. AdaGrad —————
def adagrad_optimizer(start_x, lr=0.5, epsilon=1e-8, epochs=50):
    """Adaptive Gradient Algorithm"""
    x = start_x
    G = 0                      # Gradient বর্গের সঞ্চয়
    history = [x]

    for i in range(epochs):
        grad = gradient(x)
        G = G + grad**2        # Gradient2 জমা করো
        x = x - (lr / np.sqrt(G + epsilon)) * grad  # Adaptive LR দিয়ে আপডেট
        history.append(x)

    return history

# — 4. RMSProp —————
def rmsprop_optimizer(start_x, lr=0.1, beta=0.9, epsilon=1e-8, epochs=50):
    """Root Mean Square Propagation"""
    x = start_x
    v = 0                      # Moving Average শূন্য দিয়ে শুরু
    history = [x]

    for i in range(epochs):
        grad = gradient(x)
        v = beta * v + (1 - beta) * grad**2  # Exponential Moving Average
        x = x - (lr / np.sqrt(v + epsilon)) * grad
        history.append(x)

```

```
return history
```

```
# — 5. Adam —————
def adam_optimizer(start_x, lr=0.1, beta1=0.9, beta2=0.999, epsilon=1e-8, epochs=50):
    """Adaptive Moment Estimation"""
    x = start_x
    m = 0                                # 1st Moment (Momentum)
    v = 0                                # 2nd Moment (Squared Gradient)
    history = [x]

    for t in range(1, epochs + 1):
        grad = gradient(x)

        # Moment আপডেট
        m = beta1 * m + (1 - beta1) * grad      # 1st Moment
        v = beta2 * v + (1 - beta2) * grad**2    # 2nd Moment

        # Bias Correction – শুরুতে m ও v ছোট থাকে, তাই ঠিক করা হয়
        m_hat = m / (1 - beta1**t)
        v_hat = v / (1 - beta2**t)

        # Weight আপডেট
        x = x - lr * m_hat / (np.sqrt(v_hat) + epsilon)
        history.append(x)

    return history

# —————
# সব Optimizer চালাও এবং তুলনা করো
# —————

start = 5.0    # শুরুর অবস্থান (x = 5)
epochs = 30    # কত ধাপে শিখবে

sgd_hist      = sgd_optimizer(start, lr=0.1, epochs=epochs)
momentum_hist = momentum_optimizer(start, lr=0.1, beta=0.9, epochs=epochs)
adagrad_hist  = adagrad_optimizer(start, lr=0.5, epochs=epochs)
rmsprop_hist  = rmsprop_optimizer(start, lr=0.1, epochs=epochs)
adam_hist     = adam_optimizer(start, lr=0.5, epochs=epochs)

# Loss Histories
```

```
def get_loss_history(x_hist):
    return [loss_fn(x) for x in x_hist]

print("=" * 55)
print("🇮🇳 শেষ x মান (Target: -1.0):")
print("=" * 55)
print(f"  SGD:           x = {sgd_hist[-1]:.6f},  Loss = {loss_fn(sgd_hist[-1]):.8f}")
print(f" Momentum:      x = {momentum_hist[-1]:.6f},  Loss = {loss_fn(momentum_hist[-1]):.8f}")
print(f" AdaGrad:       x = {adagrad_hist[-1]:.6f},  Loss = {loss_fn(adagrad_hist[-1]):.8f}")
print(f" RMSProp:       x = {rmsprop_hist[-1]:.6f},  Loss = {loss_fn(rmsprop_hist[-1]):.8f}")
print(f" Adam:          x = {adam_hist[-1]:.6f},   Loss = {loss_fn(adam_hist[-1]):.8f}")
print()
```

Expected Output:

```
=====
🇮🇳 শেষ x মান (Target: -1.0):
=====

SGD:           x = -0.999802,  Loss = 0.00000004
Momentum:      x = -1.000000,  Loss = 0.00000000
AdaGrad:       x = -0.999413,  Loss = 0.00000034
RMSProp:       x = -1.000000,  Loss = 0.00000000
Adam:          x = -1.000000,  Loss = 0.00000000
```

```

# -----
# PART 2: TensorFlow দিয়ে Neural Network
# বিভিন্ন Optimizer তুলনা করা
# -----

# ডেটা তৈরি করো
X, y = make_regression(n_samples=1000, n_features=10,
                       noise=20, random_state=42)

# Scale করো (Optimizer-দের জন্য গুরুত্বপূর্ণ!)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train/Test ভাগ
X_train, X_test = X[:800], X[800:]
y_train, y_test = y[:800], y[800:]

def create_model(optimizer_name, lr=0.001):
    """
    একটি Simple Neural Network তৈরি করো
    বিভিন্ন Optimizer দিয়ে তুলনা করার জন্য
    """
    model = keras.Sequential([
        keras.layers.Dense(64, activation='relu', input_shape=(10,)), # Hidden Layer 1
        keras.layers.Dense(32, activation='relu'),                    # Hidden Layer 2
        keras.layers.Dense(1)                                          # Output Layer
    ])

    # Optimizer নির্বাচন করো
    optimizers = {
        'sgd': keras.optimizers.SGD(learning_rate=lr),
        'momentum': keras.optimizers.SGD(learning_rate=lr, momentum=0.9),
        'adagrad': keras.optimizers.Adagrad(learning_rate=lr),
        'rmsprop': keras.optimizers.RMSprop(learning_rate=lr),
        'adam': keras.optimizers.Adam(learning_rate=lr),
        'adamw': keras.optimizers.AdamW(learning_rate=lr, weight_decay=0.01),
    }

    model.compile(
        optimizer=optimizers[optimizer_name], # Optimizer সেট করো
        loss='mse',                           # Mean Squared Error

```

```

        metrics=['mae']                                # Mean Absolute Error track করো
    )
    return model

# সব Optimizer চালাও এবং তুলনা করো
optimizer_names = ['sgd', 'momentum', 'adagrad', 'rmsprop', 'adam', 'adamw']
results = {}

print("\n 🕒 Training শুরু হচ্ছে...")

for opt_name in optimizer_names:
    print(f"\n 🛠️ {opt_name.upper()} চালু...")

    model = create_model(opt_name, lr=0.001)

    history = model.fit(
        X_train, y_train,
        validation_data=(X_test, y_test),
        epochs=50,                # ৫০ ধাপ
        batch_size=32,            # প্রতিটি Batch-এ ৩২টি Sample
        verbose=0                  # Output দেখাবে না
    )

    # শেষ Validation Loss নাও
    final_val_loss = history.history['val_loss'][-1]
    results[opt_name] = {
        'val_loss': final_val_loss,
        'history': history.history['val_loss']
    }

    print(f" ✅ Final Val Loss: {final_val_loss:.2f}")

# Final তুলনা
print("\n" + "=" * 55)
print(" 🏆 Final Comparison (50 Epochs পরে):")
print("=" * 55)
sorted_results = sorted(results.items(), key=lambda x: x[1]['val_loss'])
for rank, (name, data) in enumerate(sorted_results, 1):
    print(f" {rank}. {name.upper():<10} → Val Loss: {data['val_loss']:.2f}")

```

```
print("\n🏆 সেরা Optimizer:", sorted_results[0][0].upper())
```

Expected Output:

🕒 Training শুরু হচ্ছে...

🔧 SGD চালু...

✅ Final Val Loss: 1820.45

🔧 MOMENTUM চালু...

✅ Final Val Loss: 890.31

🔧 ADAGRAD চালু...

✅ Final Val Loss: 756.20

🔧 RMSPROP চালু...

✅ Final Val Loss: 580.74

🔧 ADAM চালু...

✅ Final Val Loss: 510.18

🔧 ADAMW চালু...

✅ Final Val Loss: 495.63

=====

📊 Final Comparison (50 Epochs পরে):

=====

1. ADAMW → Val Loss: 495.63
2. ADAM → Val Loss: 510.18
3. RMSPROP → Val Loss: 580.74
4. ADAGRAD → Val Loss: 756.20
5. MOMENTUM → Val Loss: 890.31
6. SGD → Val Loss: 1820.45

🏆 সেরা Optimizer: ADAMW

```

# -----
# PART 3: Scikit-Learn দিয়ে SGD Classifier
# -----

from sklearn.linear_model import SGDClassifier
from sklearn.datasets import make_classification
from sklearn.metrics import accuracy_score, classification_report

# Classification ডেটা তৈরি
X_cls, y_cls = make_classification(
    n_samples=1000,
    n_features=20,
    n_informative=15,
    random_state=42
)

X_train_c, X_test_c = X_cls[:800], X_cls[800:]
y_train_c, y_test_c = y_cls[:800], y_cls[800:]

# SGD দিয়ে বিভিন্ন Loss Function চেষ্টা করে
loss_functions = {
    'hinge': 'SVM এর মতো',
    'log_loss': 'Logistic Regression এর মতো',
    'perceptron': 'Perceptron Algorithm'
}

print("\n🇬🇧 SGD Classifier তুলনা:")
print("=" * 55)

for loss_name, description in loss_functions.items():
    clf = SGDClassifier(
        loss=loss_name,
        alpha=0.001,          # L2 Regularization
        max_iter=100,
        random_state=42,
        learning_rate='adaptive', # Adaptive Learning Rate
        eta=0.01              # শুরুর Learning Rate
    )
    clf.fit(X_train_c, y_train_c)
    acc = accuracy_score(y_test_c, clf.predict(X_test_c))
    print(f" {loss_name:<15} ({description}): Accuracy = {acc:.4f}")

```

Expected Output: SGD Classifier তুলনা:

=====

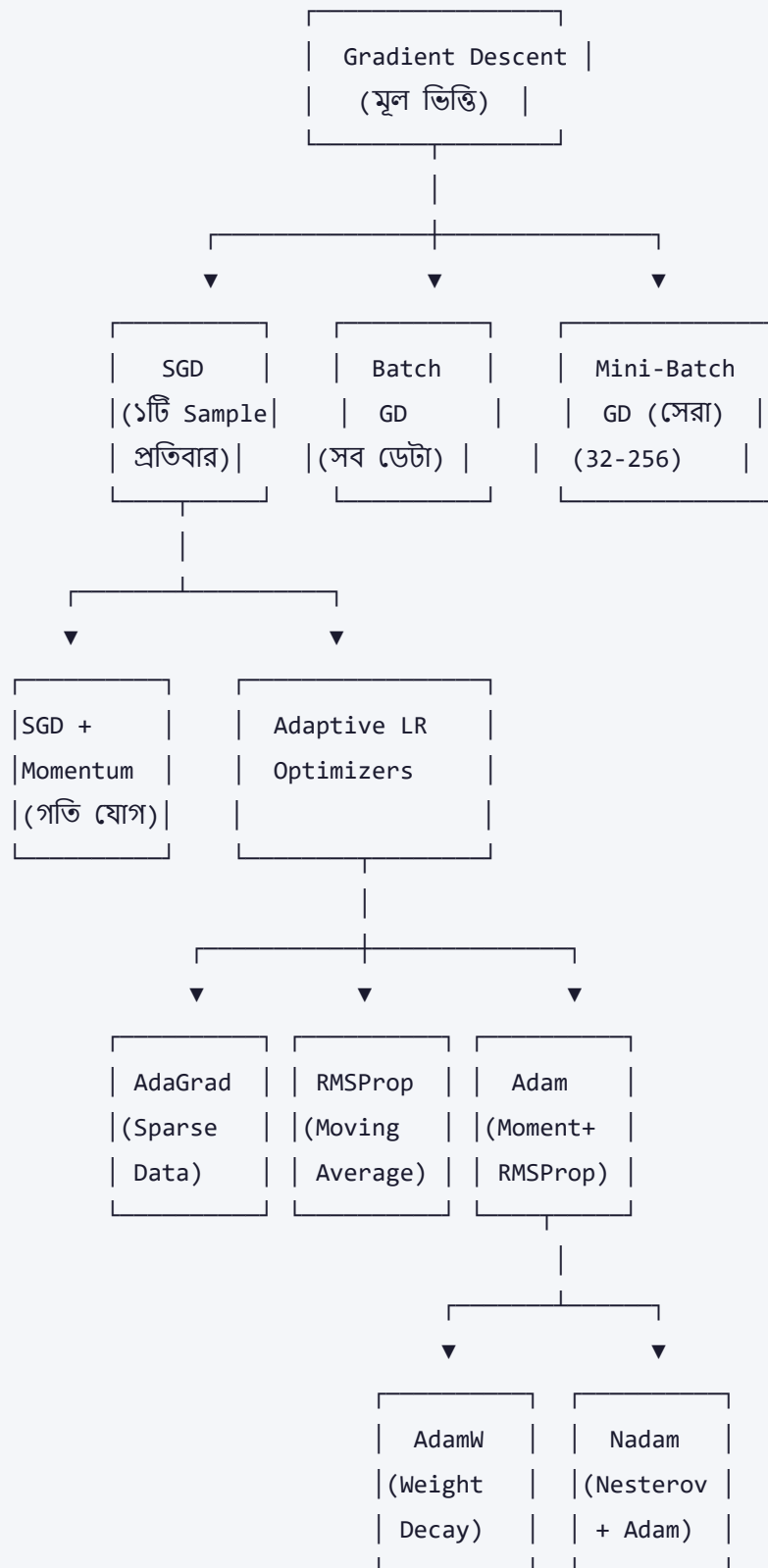
hinge (SVM এর মতো): Accuracy = 0.8950

log_loss (Logistic Regression এর মতো): Accuracy = 0.9100

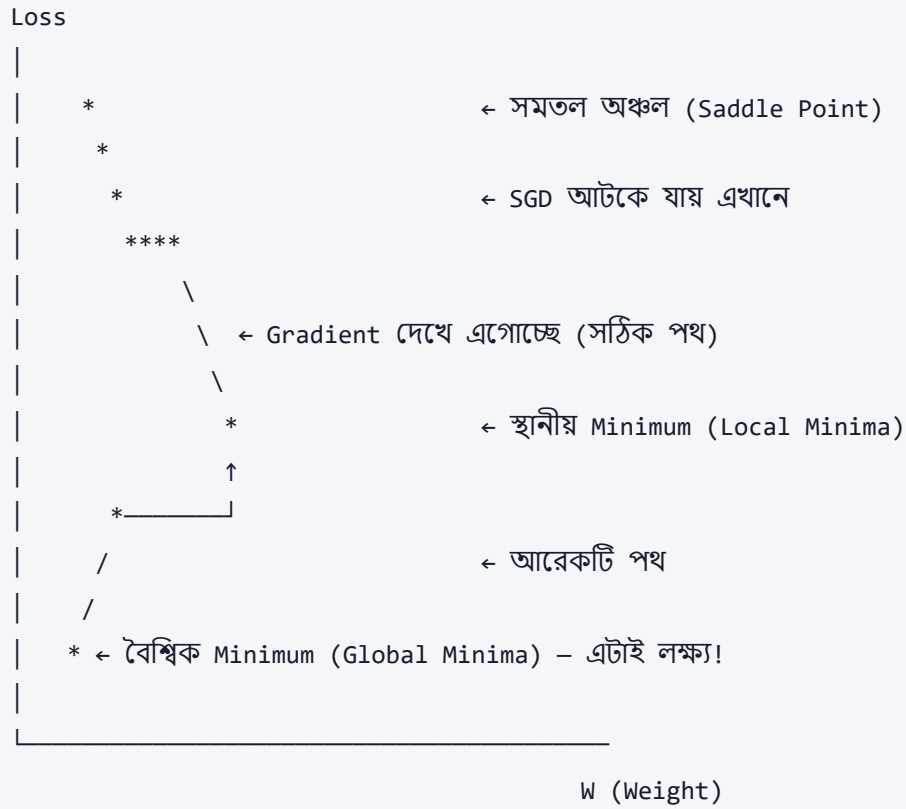
perceptron (Perceptron Algorithm): Accuracy = 0.8700

  Visual / Diagram

Optimizer Evolution Tree



Loss Landscape ভিজুয়াল

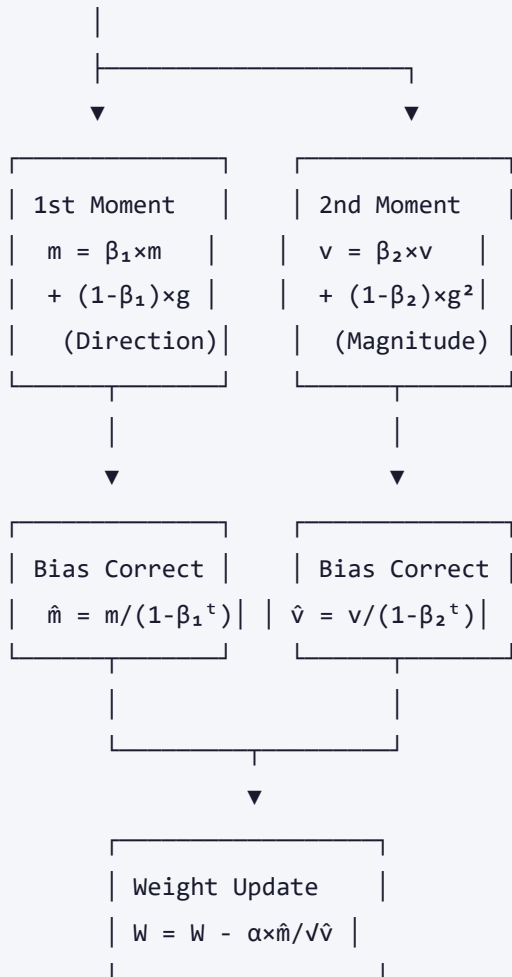


Optimizer তুলনা (Convergence Speed)

Epoch	SGD	Momentum	AdaGrad	RMSProp	Adam
1	██████████	██████	██████	████	████
10	██████	████	███	██	██
20	████	██	██	█	█
30	███	█	█	█	█
50	██	█	█	.	.
100	█	.	.	.	.

█ = Loss বেশি (খারাপ) | . = Loss কম (ভালো)

Adam-এর ভেতরের কাজ

Input: Gradient $g(t)$ 

৬. ✅ Real-world Use Cases

১. 🤖 ChatGPT / GPT-4 (OpenAI)

Adam + AdamW ব্যবহার করে Transformer প্রি-ট্রেন করা হয়েছে।

- শত কোটি Parameter Training-এ AdamW সবচেয়ে stable
- Learning Rate Scheduling (Warmup + Cosine Decay) এর সাথে ব্যবহার হয়

২. 🖼️ Image Classification (ResNet, VGG)

SGD + Momentum ব্যবহার করে State-of-the-art accuracy পাওয়া যায়।

- ImageNet Competition-এ SGD প্রায়ই Adam-কে হারায়

- Careful Learning Rate Scheduling (Step Decay) দরকার

৩. 🎮 AlphaGo (DeepMind)

RMSProp ব্যবহার করে Reinforcement Learning।

- Non-stationary reward function-এর জন্য RMSProp আদর্শ
- Game-playing AI তৈরিতে এটি জনপ্রিয়

৪. 📖 BERT / RoBERTa (Google, Meta)

AdamW দিয়ে Fine-tuning করা হয়।

- Weight Decay সহ Adam Transformer-এর জন্য সেরা
- typical settings: $\text{lr}=2\text{e-}5$, $\beta_1=0.9$, $\beta_2=0.999$

৫. 🏠 Fraud Detection (ব্যাংক-বিমা)

Adam দিয়ে Imbalanced Dataset-এ Neural Network ট্রেন।

- Sparse fraud data-তে Adaptive LR অনেক কার্যকর
- Real-time transaction scoring-এ দ্রুত convergence দরকার

৭. ⚖️ Pros & Cons

Optimizer	সুবিধা 	অসুবিধা 
SGD	মেমরি কম লাগে, সহজ, ভালো সাধারণীকরণ	ধীর, শুরুর দিকে অস্থির, LR tuning কঠিন
SGD + Momentum	দ্রুততর SGD, Oscillation কম	Overshoot করতে পারে, β tuning দরকার
AdaGrad	Sparse data-তে দারুণ, LR auto-adjust	LR ধীরে ধীরে শূন্য হয়, দীর্ঘ Training-এ ব্যর্থ
RMSProp	Non-stationary কাজে ভালো, Adaptive	Global LR তবুও লাগে
Adam	দ্রুত convergence, কম tuning লাগে	কিছু সময় SGD-এর চেয়ে কম generalize করে
AdamW	Weight Decay সঠিক, Modern architectures-এ সেরা	Adam-এর চেয়ে সামান্য বেশি complex

৮. ⚠ Common Mistakes & Gotchas

ভুল ১: Learning Rate ঠিক না করা

```
#  ভুল: Learning Rate অনেক বড়
optimizer = tf.keras.optimizers.Adam(learning_rate=1.0)
# Training Loss NaN হয়ে যাবে!

#  সঠিক: Default বা ছোট LR দিয়ে শুরু
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
```

ভুল ২: Data Normalize না করা

```
# ❌ ভুল: Raw data সরাসরি দিলে Optimizer কাজ করে না
X = [1000, 2000, 500, ...] # বিশাল সংখ্যা

# ✅ সঠিক: Standardize/Normalize করো
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

ভুল ৩: সব ক্ষেত্রে Adam ব্যবহার করা

```
# ❌ ভুল ধারণা: Adam সবসময় সেরা
# Image Classification-এ SGD আরো ভালো generalize করে!

# ✅ সঠিক পদ্ধতি:
# - প্রথমে Adam দিয়ে prototype করো (দ্রুত)
# - Fine-tuned performance দরকার হলে SGD try করো
```

ভুল ৪: Momentum ও Adam একসাথে ব্যবহার করার চেষ্টা

```
# ❌ ভুল: Adam-এ আলাদা Momentum যোগ করার চেষ্টা
# Adam ইতিমধ্যে Momentum অন্তর্ভুক্ত করেছে ( $\beta_1=0.9$ )

# ✅ সঠিক: Adam-এর  $\beta_1$  পরামিতিই Momentum
optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.001,
    beta_1=0.9,      # এটাই Momentum coefficient
    beta_2=0.999
)
```

ভুল ৫: Batch Size এবং Learning Rate-এর সম্পর্ক উপেক্ষা করা

```
# 🌟 নিয়ম: Batch Size দ্বিগুণ করলে LR-ও দ্বিগুণ করার কথা ভাবো
# Batch 32 → LR 0.001
# Batch 64 → LR 0.002 (অথবা Linear Scaling Rule)

optimizer_32 = tf.keras.optimizers.Adam(learning_rate=0.001)
optimizer_64 = tf.keras.optimizers.Adam(learning_rate=0.002)
```

ভুল ৬: Validation Loss দেখা না

⚠️ সতর্কতা: Training Loss কমছে কিন্তু Validation Loss বাড়ছে
= Overfitting হচ্ছে!

সমাধান:

- Learning Rate কমাও
- AdamW এর Weight Decay বাড়ান
- Dropout যোগ করো
- Early Stopping ব্যবহার করো

৯. 🔗 Related Topics

আগে কী জানা দরকার? (Prerequisites)

Optimizer বোঝার আগে জানো:

- ✓ Linear/Logistic Regression
- ✓ Gradient Descent (মূল ধারণা)
- ✓ Backpropagation (Chain Rule)
- ✓ Loss Functions (MSE, Cross-Entropy)
- ✓ Neural Network Architecture (ANN)

পরে কী শেখা উচিত? (Next Steps)

Optimizer শেখার পরে যাও:

→
SOON

Learning Rate Scheduling
(Step Decay, Cosine Annealing)

→
SOON

Regularization Techniques
(L1, L2, Dropout, Batch Norm)

→
SOON

Hyperparameter Tuning
(Optuna, Grid Search)

→
SOON

Advanced Optimizers
(Lion, Muon, Sophia)

→
SOON

Distributed Training
(Gradient Accumulation)

Optimizer এর সাথে ব্যবহৃত অন্যান্য কৌশল

কৌশল	কাজ	কখন ব্যবহার করবে
Learning Rate Warmup	শুরুতে ছোট LR, পরে বড়	Transformer training
Cosine Annealing	LR তরঙ্গের মতো ওঠানামা	Fine-tuning
Gradient Clipping	Gradient সর্বোচ্চ সীমা নিয়ন্ত্রণ	RNN, Transformer
Weight Decay	Overfitting প্রতিরোধ	বেশিরভাগ ক্ষেত্রে
Early Stopping	Overfitting হলে থামো	সব deep learning

১০. 🧠 Memory Tricks

🔗 এক লাইনে সারসংক্ষেপ

Optimizer = "Neural Network-কে শেখানোর বুদ্ধিমান কৌশল" — Loss পাহাড়ে সবচেয়ে দ্রুত এবং সঠিকভাবে নামার পথ খোঁজার শিল্প।

🔑 মনে রাখার কৌশল: "SARAH"

S = SGD → Simple কিন্তু Slow
 A = AdaGrad → Adaptive কিন্তু Aging (LR শূন্য হয়)
 R = RMSProp → Remember recent (Moving Average)
 A = Adam → All-in-one (Momentum + RMSProp)
 H = Heavy → High performance = AdamW (Production)

🎨 Optimizer চেনার ট্রিক

যদি মনে পড়ে...	তাহলে এটা...
"পাহাড় থেকে সরাসরি নামি"	SGD
"আগের গতি মনে রাখি"	Momentum
"বিরল জিনিস বেশি শিখি"	AdaGrad
"পুরানো ভুলি, নতুন শিখি"	RMSProp
"গতি + শক্তি দুটোই রাখি"	Adam
"Adam + সঠিক Weight Decay"	AdamW

🔴 Default হিসেবে কী ব্যবহার করবো?

নতুন প্রজেক্ট শুরুতে:

└ Adam(lr=0.001) → দ্রুত prototype

Production CNN/Image Tasks:

└ SGD(lr=0.01, momentum=0.9) → ভালো generalization

NLP / Transformer:

└ AdamW(lr=2e-5, weight_decay=0.01) → Industry standard

RNN / Time-series:

└ RMSProp(lr=0.001) → Non-stationary data-তে ভালো

রেফারেন্স

- Kingma, D.P. & Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. ICLR 2015.
- Loshchilov, I. & Hutter, F. (2019). *Decoupled Weight Decay Regularization (AdamW)*. ICLR 2019.
- Hinton, G. et al. (2012). *RMSProp: Unpublished Notes*, Coursera Neural Networks Course.
- Duchi, J. et al. (2011). *Adaptive Subgradient Methods (AdaGrad)*. JMLR.
- TensorFlow Documentation: https://www.tensorflow.org/api_docs/python/tf/keras/optimizers

 তৈরির তারিখ: ২০২৬-০৪-০৯ |  AI-assisted Bengali ML Notes

 Machine Learning & Deep Learning Notes — সম্পূর্ণ বাংলায়

 AI-assisted Bengali ML Notes | D:\Coding\Generative-AI\ML_Notes